

Walmart Black Friday — Customer Purchase Behavior Analysis

Business Case: Confidence Intervals & Central Limit Theorem

Author: Shivesh (Ethan) Sahu **Date:** June 2026

Project Overview

About Walmart

Walmart is an American multinational retail corporation operating a chain of supercenters, discount departmental stores, and grocery stores across the United States, with over **100 million customers worldwide**.

Business Problem

The Management team at Walmart Inc. wants to analyze customer purchase behavior — specifically **purchase amounts** — against customer demographics to make better business decisions.

Central Question: *Do women spend more on Black Friday than men? (Assuming a population of 50 million male and 50 million female customers)*

Approach

Using the **Central Limit Theorem (CLT)** and **Confidence Intervals**, we will:

- Determine whether spending habits differ between male and female customers
- Extend the analysis to marital status (Married vs Unmarried) and age groups
- Provide actionable, data-driven recommendations to Walmart leadership

Table of Contents

1. Data Import & Initial Exploration
2. Missing Values & Outlier Detection

3. Exploratory Data Analysis (EDA)
 - Univariate Analysis
 - Bivariate Analysis
 - Correlation Analysis
4. Central Limit Theorem & Confidence Interval Analysis
 - 4.1 Gender: Male vs Female
 - 4.2 Marital Status: Married vs Unmarried
 - 4.3 Age Group Analysis
5. Final Insights
6. Business Recommendations

1. Importing Libraries and Loading the Dataset

We begin by importing the necessary libraries and loading the Walmart Black Friday dataset. We'll also do an initial inspection to understand the shape and structure of the data before any analysis.

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

print("pandas:", pd.__version__)
print("numpy:", np.__version__)
print("seaborn:", sns.__version__)
print("All good – let's go!!!")
```

```
pandas: 3.0.3
numpy: 2.4.6
seaborn: 0.13.2
All good – let's go!!!
```

```
In [16]: df = pd.read_csv('walmart_data.csv')
print("Since the dataset is correctly loaded.")
print("\nLets have a look at the first 5 rows")
```

Since the dataset is correctly loaded.

Lets have a look at the first 5 rows

```
In [15]: print("\nFirst 5 rows")
print(df.head())
```

First 5 rows

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	\
0	1000001	P00069042	F	0-17	10	A	
1	1000001	P00248942	F	0-17	10	A	
2	1000001	P00087842	F	0-17	10	A	
3	1000001	P00085442	F	0-17	10	A	
4	1000002	P00285442	M	55+	16	C	

	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	2	0	3	837
1	2	0	1	1520
2	2	0	12	142
3	2	0	12	105
4	4+	0	8	796

```
In [14]: print("\nJust out of curiosity, lets see some random samples of 5 rows\n")
print(df.sample(5, random_state=42))
```

Just out of curiosity, lets see some random samples of 5 rows

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	\
84432	1001062	P00351342	M	51-55	19	C	
72724	1005172	P00237542	F	36-45	11	B	
197032	1000415	P00116242	F	36-45	0	A	
353704	1000496	P00016742	M	18-25	4	A	
91198	1002010	P00002242	M	18-25	4	B	

	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
84432	3	0	1	19142
72724	1	0	1	15513
197032	2	1	8	7802
353704	3	0	1	15455
91198	0	0	6	4492

```
In [12]: # The Shape of the dataset
print("Shape", df.shape)
print(f"\nThe dataset has {df.shape[0]:,} rows and {df.shape[1]} columns")
```

Shape (550068, 10)

The dataset has 550,068 rows and 10 columns

Data Types and Structures

```
In [17]: print("Column Data Types")
print(df.dtypes)

print("\nDataset Info")
df.info()
```

Column Data Types

```
User_ID          int64
Product_ID       str
Gender           str
Age             str
Occupation       int64
City_Category    str
Stay_In_Current_City_Years  str
Marital_Status   int64
Product_Category int64
Purchase         int64
dtype: object
```

Dataset Info

```
<class 'pandas.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	User_ID	550068 non-null	int64
1	Product_ID	550068 non-null	str
2	Gender	550068 non-null	str
3	Age	550068 non-null	str
4	Occupation	550068 non-null	int64
5	City_Category	550068 non-null	str
6	Stay_In_Current_City_Years	550068 non-null	str
7	Marital_Status	550068 non-null	int64
8	Product_Category	550068 non-null	int64
9	Purchase	550068 non-null	int64

```
dtypes: int64(5), str(5)
```

```
memory usage: 42.0 MB
```

Statistical Summary

```
In [18]: print("Statistical Summary (Numerical Columns)")
display(df.describe())

print("\nStatistical Summary (Categorical Columns)")
display(df.describe(include='object'))
```

Statistical Summary (Numerical Columns)

	User_ID	Occupation	Marital_Status	Product_Category	Purchase
count	5.500680e+05	550068.000000	550068.000000	550068.000000	550068
mean	1.003029e+06	8.076707	0.409653	5.404270	9263
std	1.727592e+03	6.522660	0.491770	3.936211	5023
min	1.000001e+06	0.000000	0.000000	1.000000	12
25%	1.001516e+06	2.000000	0.000000	1.000000	5823
50%	1.003077e+06	7.000000	0.000000	5.000000	8047
75%	1.004478e+06	14.000000	1.000000	8.000000	12054
max	1.006040e+06	20.000000	1.000000	20.000000	23961

Statistical Summary (Categorical Columns)

	Product_ID	Gender	Age	City_Category	Stay_In_Current_City_Years
count	550068	550068	550068	550068	550068
unique	3631	2	7	3	5
top	P00265242	M	26-35	B	1
freq	1880	414259	219587	231173	193821

1.4 Key Observations from Initial Exploration

Dataset Structure:

- The dataset contains **550,068 transactions** across **10 features** — one row = one purchase event, not one customer. A single customer can appear multiple times.
- Memory footprint: 42 MB — manageable in-memory with pandas.

Data Types:

- 5 numeric columns: `User_ID`, `Occupation`, `Marital_Status`, `Product_Category`, `Purchase`
- 5 categorical columns: `Product_ID`, `Gender`, `Age`, `City_Category`, `Stay_In_Current_City_Years`
- `Marital_Status` is stored as int64 (0 = Single, 1 = Married) but is conceptually categorical.

Missing Values (preliminary):

- All 10 columns show **550,068 non-null** entries → no missing values detected

at first glance. This will be confirmed formally in the next section.

Purchase Amount — most important column:

- Mean: **\$9,264** vs Median: **\$8,047** → Mean > Median by ~\$1,217
- This tells us the distribution is **right-skewed** — a small number of very high-value purchases are pulling the average upward.
- Range: 12to23,961 — wide spread, confirming the presence of outliers.
- This right skew is exactly why we need the **Central Limit Theorem** later — it allows us to work with sample means which will be normally distributed, even though the raw purchase data is skewed.

Gender:

- Male transactions: **414,259** ($\approx 75\%$) vs Female: **135,809** ($\approx 25\%$)
- Important: this is a count of *transactions*, not customers. More male transactions does NOT mean women spend less — we must compare *average purchase amounts*, not totals.

Age & City:

- Most active age group: **26–35** (219,587 transactions)
- Most active city type: **B** (231,173 transactions)
- `Stay_In_Current_City_Years` has 5 unique values (0, 1, 2, 3, 4+)

```
In [19]: # Convert categorical columns from string to 'category' dtype
# This improves memory efficiency and makes groupby operations faster

cat_cols = ['Gender', 'Age', 'City_Category',
            'Stay_In_Current_City_Years', 'Marital_Status']

for col in cat_cols:
    df[col] = df[col].astype('category')

print("Updated Data Types:")
print(df.dtypes)
print(f"\nMemory after conversion: {df.memory_usage(deep=True).sum()} /
```

```
Updated Data Types:
User_ID                int64
Product_ID            str
Gender                category
Age                  category
Occupation            int64
City_Category         category
Stay_In_Current_City_Years  category
Marital_Status        category
Product_Category      int64
Purchase              int64
dtype: object
```

Memory after conversion: 49.8 MB

2. Missing Values and Outlier Detection

Missing Values

```
In [20]: missing_count = df.isnull().sum()
missing_percentage = (df.isnull().sum()/len(df))*100

missing_df = pd.DataFrame({'Missing Values': missing_count, 'Missing P

print("\n Missing Value Report")
display(missing_df.sort_values(by='Missing Percentage', ascending=False

if missing_count.sum() == 0:
    print("\nNo missing values")
else:
    print(f"\n Total Missing Values: {missing_count.sum()}")
```

Missing Value Report

	Missing Values	Missing Percentage
User_ID	0	0.0
Product_ID	0	0.0
Gender	0	0.0
Age	0	0.0
Occupation	0	0.0
City_Category	0	0.0
Stay_In_Current_City_Years	0	0.0
Marital_Status	0	0.0
Product_Category	0	0.0
Purchase	0	0.0

No missing values

Outlier Detection

```
In [24]: # Method 1:
# Mean VS Median Comparison
mean_purchase = df['Purchase'].mean()
median_purchase = df['Purchase'].median()
diff = mean_purchase - median_purchase

print("\nMean VS Median Purchase")
print(f"Mean Purchase: {mean_purchase:.2f}")
print(f"Median Purchase: {median_purchase:.2f}")
print(f"Diff: {diff:.2f}")
print(f"\nMean is {(diff/mean_purchase*100):.1f}% higher than median,
```

Mean VS Median Purchase
Mean Purchase: 9263.97
Median Purchase: 8047.00
Diff: 1216.97

Mean is 13.1% higher than median, therefore Right-Skewed Distribution

```
In [30]: # Method 2:
# IQR Method to quantify outliers
Q1 = df['Purchase'].quantile(0.25)
Q3 = df['Purchase'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5*IQR
upper_bound = Q3 + 1.5*IQR
```



```

outliers = df[(df['Purchase'] > upper_bound) | (df['Purchase'] < lower_bound)]

print("=== IQR Outlier Analysis ===")
print(f"Q1 (25th percentile): ${Q1:,.2f}")
print(f"Q3 (75th percentile): ${Q3:,.2f}")
print(f"IQR: ${IQR:,.2f}")
print(f"\nLower Bound: ${lower_bound:,.2f}")
print(f"Upper Bound: ${upper_bound:,.2f}")
print(f"\nOutlier transactions: {len(outliers):,} ({len(outliers)/len(df):.1%})")
print(f"Outlier purchase range: ${outliers['Purchase'].min():,} to ${outliers['Purchase'].max():,}")

```

=== IQR Outlier Analysis ===

Q1 (25th percentile): \$5,823.00

Q3 (75th percentile): \$12,054.00

IQR: \$6,231.00

Lower Bound: \$-3,523.50

Upper Bound: \$21,400.50

Outlier transactions: 2,677 (0.5% of data)

Outlier purchase range: \$21,401 to \$23,961

```

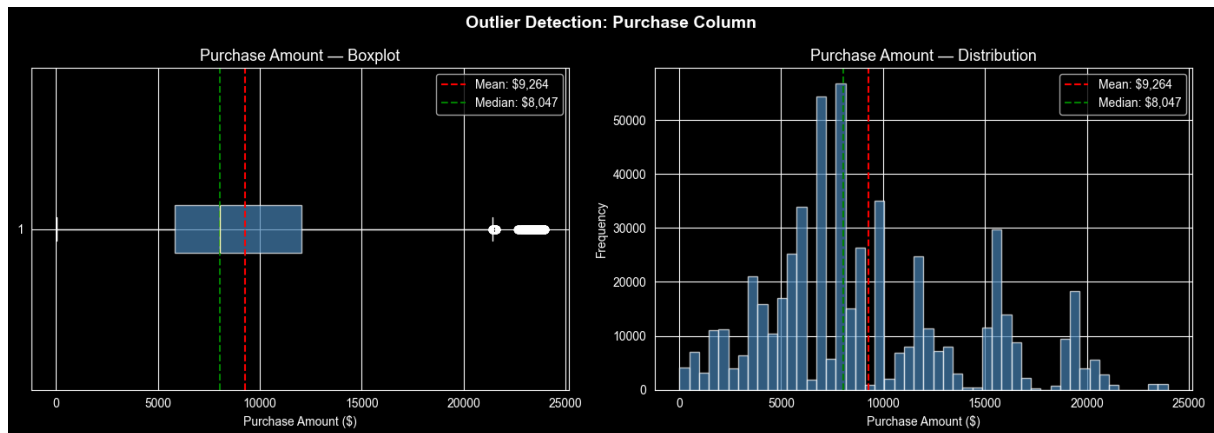
In [33]: # Method 3:
# BoxPlot -> Visual Conformation
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Boxplot
axes[0].boxplot(df['Purchase'], vert=False, patch_artist=True,
                boxprops=dict(facecolor='steelblue', alpha=0.7))
axes[0].set_title('Purchase Amount - Boxplot', fontsize=13)
axes[0].set_xlabel('Purchase Amount ($)')
axes[0].axvline(mean_purchase, color='red', linestyle='--', label=f'Mean')
axes[0].axvline(median_purchase, color='green', linestyle='--', label=f'Median')
axes[0].legend()

# Distribution plot
axes[1].hist(df['Purchase'], bins=50, color='steelblue', alpha=0.7, edgecolor='black')
axes[1].axvline(mean_purchase, color='red', linestyle='--', label=f'Mean')
axes[1].axvline(median_purchase, color='green', linestyle='--', label=f'Median')
axes[1].set_title('Purchase Amount - Distribution', fontsize=13)
axes[1].set_xlabel('Purchase Amount ($)')
axes[1].set_ylabel('Frequency')
axes[1].legend()

plt.suptitle('Outlier Detection: Purchase Column', fontsize=14, fontweight='bold')
plt.tight_layout()

```



Observations — Missing Values & Outliers

Missing Values:

- Zero missing values across all 550,068 rows and 10 columns — the dataset is complete.
- No imputation or row-dropping required.

Outliers in Purchase Amount (IQR Method):

- $Q1 = 5,823$ | $Q3 = 12,054$ | $IQR = 6,231$
- Upper Bound = 21,400.50 (lower bound is negative — no lower outliers possible)
- 2,677 transactions (0.5%) identified as outliers, ranging from 21,401 to 23,961
- All outliers are on the upper end only — confirming a right-skewed distribution

Decision: Outliers are RETAINED. In retail transaction data, high-value purchases are legitimate business events — bulk buyers, premium products, gifting season. Removing them would distort the population-level CLT and Confidence Interval analysis that follows. The Central Limit Theorem will handle this skewness naturally by normalizing the distribution of sample means regardless of the original distribution shape.

Exploratory Data Analysis (EDA)

Non-Graphical Analysis - Value Counts

```
In [42]: print("Unique Values Per Column")
print(df.nunique())
```

```

print("\nValue Counts Per Categorical Column\n")
cat_display = ['Gender', 'Age', 'City_Category', 'Stay_In_Current_City_Years']

for col in cat_display:
    print(f"---{col}---\n")
    vc = df[col].value_counts()
    pct = (df[col].value_counts(normalize=True) * 100).round(1)
    result = pd.DataFrame({'Count': vc, 'Percentage %': pct})
    print(result)
    print()

```

Unique Values Per Column

```

User_ID          5891
Product_ID      3631
Gender           2
Age             7
Occupation      21
City_Category    3
Stay_In_Current_City_Years  5
Marital_Status   2
Product_Category 20
Purchase        18105
dtype: int64

```

Value Counts Per Categorical Column

---Gender---

	Count	Percentage %
Gender		
M	414259	75.3
F	135809	24.7

---Age---

	Count	Percentage %
Age		
26-35	219587	39.9
36-45	110013	20.0
18-25	99660	18.1
46-50	45701	8.3
51-55	38501	7.0
55+	21504	3.9
0-17	15102	2.7

---City_Category---

	Count	Percentage %
City_Category		
B	231173	42.0
C	171175	31.1
A	147720	26.9

---Stay_In_Current_City_Years---

	Count	Percentage %
Stay_In_Current_City_Years		
1	193821	35.2
2	101838	18.5
3	95285	17.3
4+	84726	15.4
0	74398	13.5

---Marital_Status---

	Count	Percentage %
Marital_Status		
0	324731	59.0
1	225337	41.0

Univariate Analysis - Categorical Variables

```
In [43]: fig, axes = plt.subplots(2, 3, figsize=(18, 11))

# Gender
sns.countplot(data=df, x='Gender', ax=axes[0,0],
               palette={'M':'steelblue', 'F':'coral'})
axes[0,0].set_title('Gender Distribution', fontsize=13)
axes[0,0].set_xlabel('Gender (M = Male, F = Female)')
for p in axes[0,0].patches:
    axes[0,0].annotate(f'{int(p.get_height()):,}',
                       (p.get_x() + p.get_width()/2, p.get_height()),
                       ha='center', va='bottom', fontsize=10)

# Age
age_order = ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
sns.countplot(data=df, x='Age', ax=axes[0,1],
               palette='Blues_d', order=age_order)
axes[0,1].set_title('Age Group Distribution', fontsize=13)
axes[0,1].tick_params(axis='x', rotation=45)

# City Category
sns.countplot(data=df, x='City_Category', ax=axes[0,2], palette='Set2')
axes[0,2].set_title('City Category Distribution', fontsize=13)
axes[0,2].set_xlabel('City Category (A = Tier 1, B = Tier 2, C = Tier 3)')

# Stay in City Years
sns.countplot(data=df, x='Stay_In_Current_City_Years', ax=axes[1,0],
               palette='Blues_d')
axes[1,0].set_title('Years in Current City', fontsize=13)

# Marital Status
sns.countplot(data=df, x='Marital_Status', ax=axes[1,1],
               palette={'0':'steelblue', '1':'coral'})
```

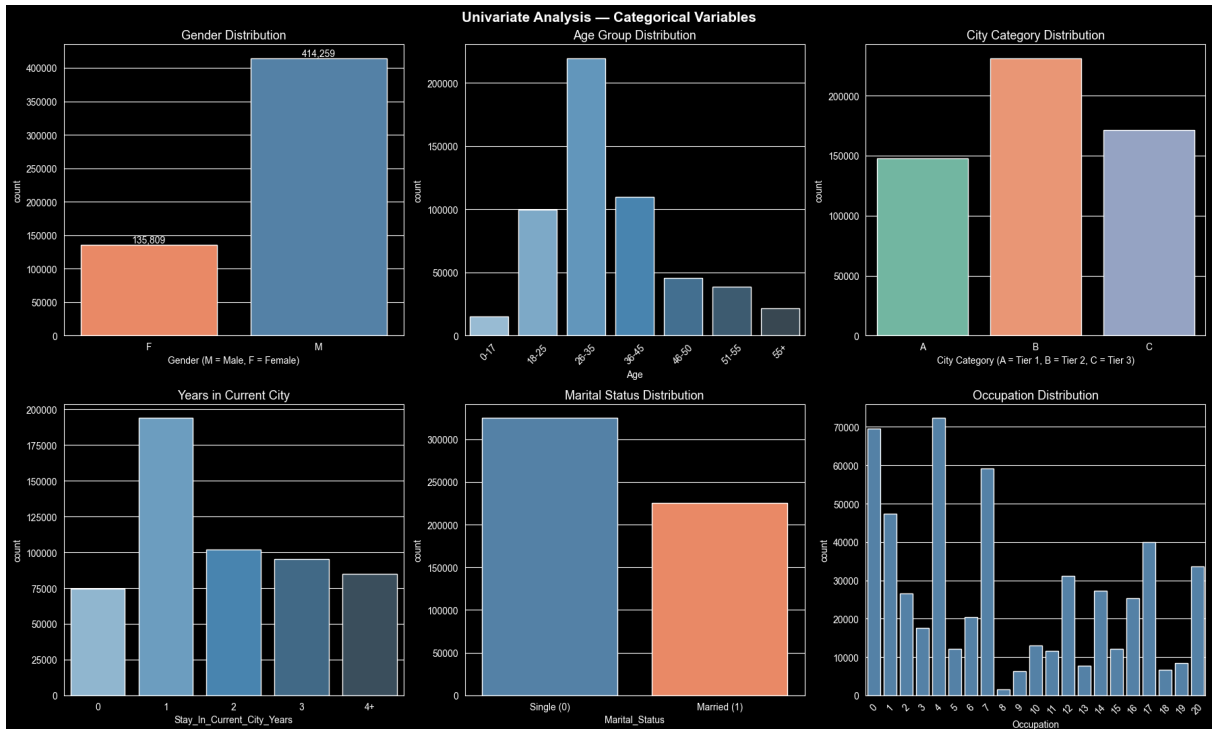
```

axes[1,1].set_title('Marital Status Distribution', fontsize=13)
axes[1,1].set_xticklabels(['Single (0)', 'Married (1)'])

# Occupation
sns.countplot(data=df, x='Occupation', ax=axes[1,2], color='steelblue')
axes[1,2].set_title('Occupation Distribution', fontsize=13)
axes[1,2].tick_params(axis='x', rotation=45)

plt.suptitle('Univariate Analysis – Categorical Variables',
             fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()

```



Univariate Analysis - Purchase Amount (Continuous Variable)

```

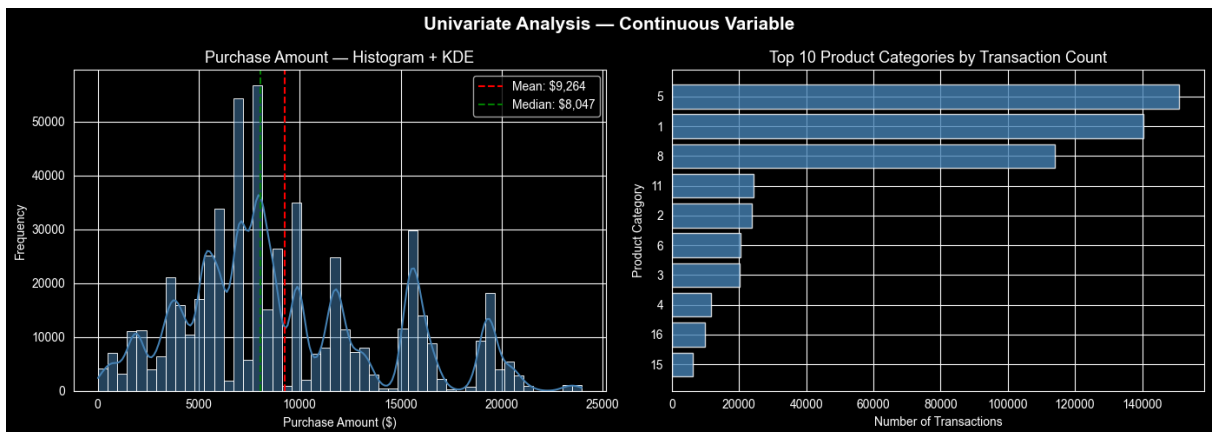
In [44]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram with KDE (distplot replacement)
sns.histplot(df['Purchase'], bins=50, kde=True,
             color='steelblue', ax=axes[0])
axes[0].axvline(df['Purchase'].mean(), color='red', linestyle='--',
               label=f'Mean: ${df["Purchase"].mean():.0f}')
axes[0].axvline(df['Purchase'].median(), color='green', linestyle='--',
               label=f'Median: ${df["Purchase"].median():.0f}')
axes[0].set_title('Purchase Amount – Histogram + KDE', fontsize=13)
axes[0].set_xlabel('Purchase Amount ($)')
axes[0].set_ylabel('Frequency')
axes[0].legend()

```

```
# Product Category distribution (top 10)
top_cats = df['Product_Category'].value_counts().head(10)
axes[1].barh(top_cats.index.astype(str), top_cats.values,
              color='steelblue', alpha=0.8)
axes[1].set_title('Top 10 Product Categories by Transaction Count', fo
axes[1].set_xlabel('Number of Transactions')
axes[1].set_ylabel('Product Category')
axes[1].invert_yaxis()

plt.suptitle('Univariate Analysis – Continuous Variable',
             fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```



Bivariate Analysis – Demographic VS Purchase Amount

```
In [45]: fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# Gender vs Purchase
sns.boxplot(data=df, x='Gender', y='Purchase', ax=axes[0,0],
            palette={'M':'steelblue', 'F':'coral'})
axes[0,0].set_title('Purchase by Gender', fontsize=13)

# Age vs Purchase
sns.boxplot(data=df, x='Age', y='Purchase', ax=axes[0,1],
            palette='Blues_d', order=age_order)
axes[0,1].set_title('Purchase by Age Group', fontsize=13)
axes[0,1].tick_params(axis='x', rotation=45)

# City Category vs Purchase
sns.boxplot(data=df, x='City_Category', y='Purchase',
            ax=axes[0,2], palette='Set2')
axes[0,2].set_title('Purchase by City Category', fontsize=13)

# Marital Status vs Purchase
sns.boxplot(data=df, x='Marital_Status', y='Purchase',
            ax=axes[1,0], palette='Set2')
axes[1,0].set_title('Purchase by Marital Status', fontsize=13)
axes[1,0].set_xticklabels(['Single', 'Married'])
```

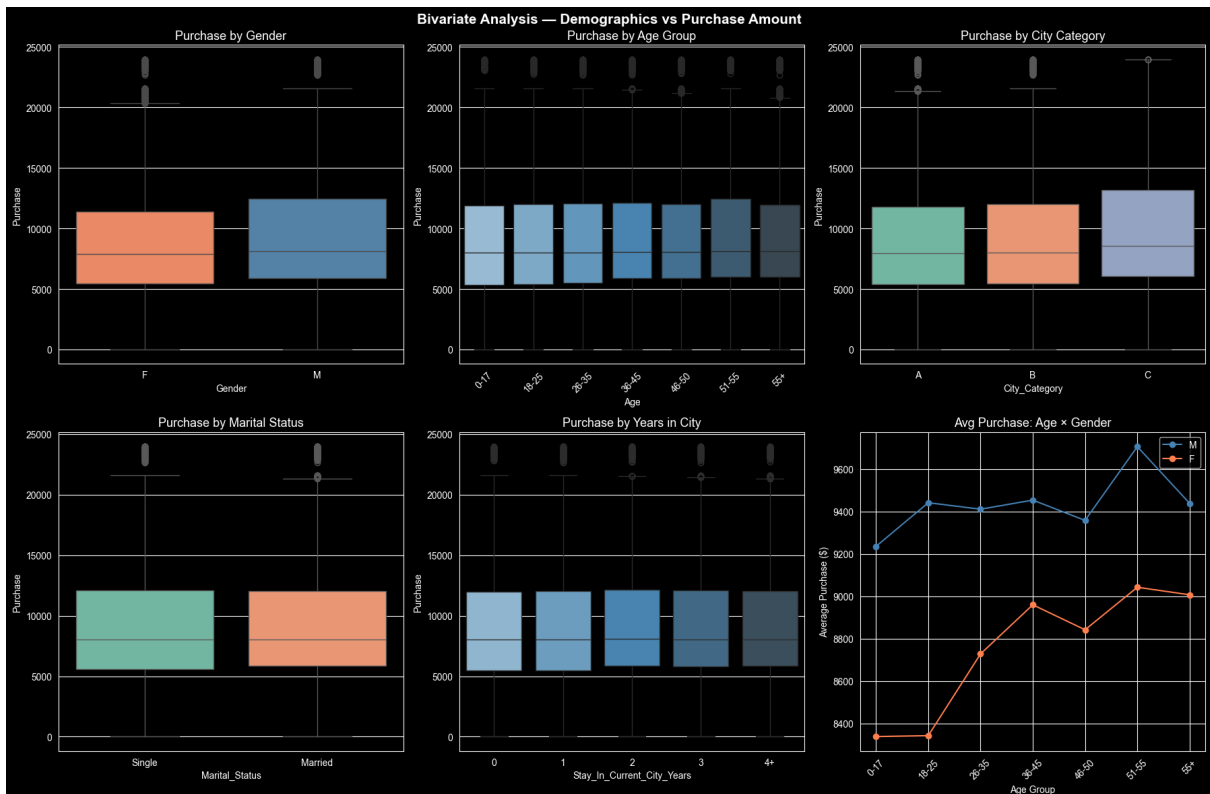
```

# Stay in City vs Purchase
sns.boxplot(data=df, x='Stay_In_Current_City_Years', y='Purchase',
            ax=axes[1,1], palette='Blues_d')
axes[1,1].set_title('Purchase by Years in City', fontsize=13)

# Gender + Age vs Purchase (mean)
gender_age = df.groupby(['Age', 'Gender'])['Purchase'].mean().reset_index()
for gender, color in [('M', 'steelblue'), ('F', 'coral')]:
    subset = gender_age[gender_age['Gender'] == gender]
    axes[1,2].plot(subset['Age'], subset['Purchase'],
                  marker='o', label=gender, color=color)
axes[1,2].set_title('Avg Purchase: Age × Gender', fontsize=13)
axes[1,2].set_xlabel('Age Group')
axes[1,2].set_ylabel('Average Purchase ($)')
axes[1,2].legend()
axes[1,2].tick_params(axis='x', rotation=45)

plt.suptitle('Bivariate Analysis – Demographics vs Purchase Amount',
            fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()

```



Correlation Analysis - Heat Map

```

In [46]: # Heatmap
num_cols = ['Occupation', 'Marital_Status', 'Product_Category', 'Purch

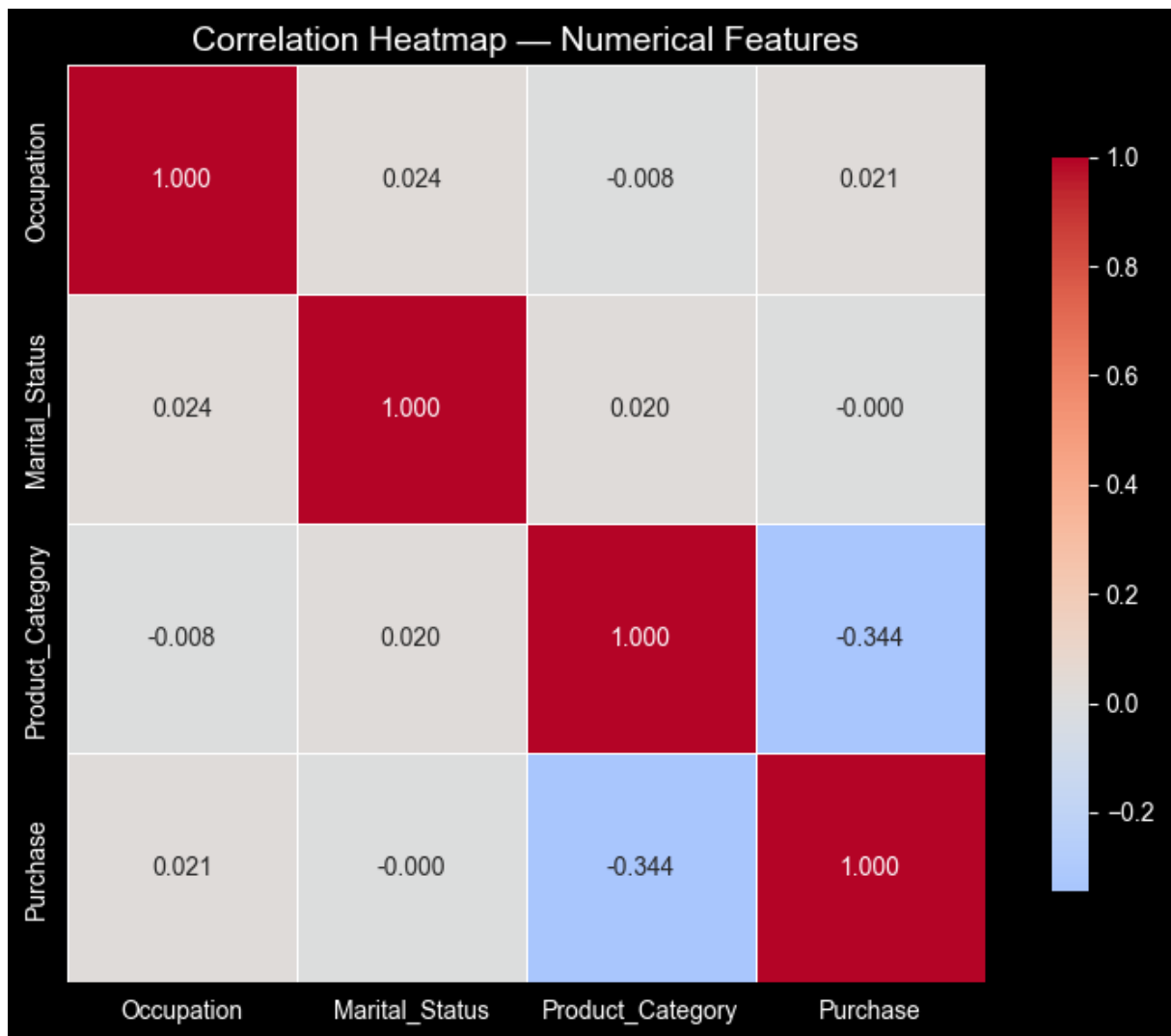
plt.figure(figsize=(8, 6))

```

```

sns.heatmap(df[num_cols].corr(), annot=True, fmt='.3f',
            cmap='coolwarm', center=0, square=True,
            linewidths=0.5, cbar_kws={'shrink': 0.8})
plt.title('Correlation Heatmap — Numerical Features', fontsize=14)
plt.tight_layout()
plt.show()

```

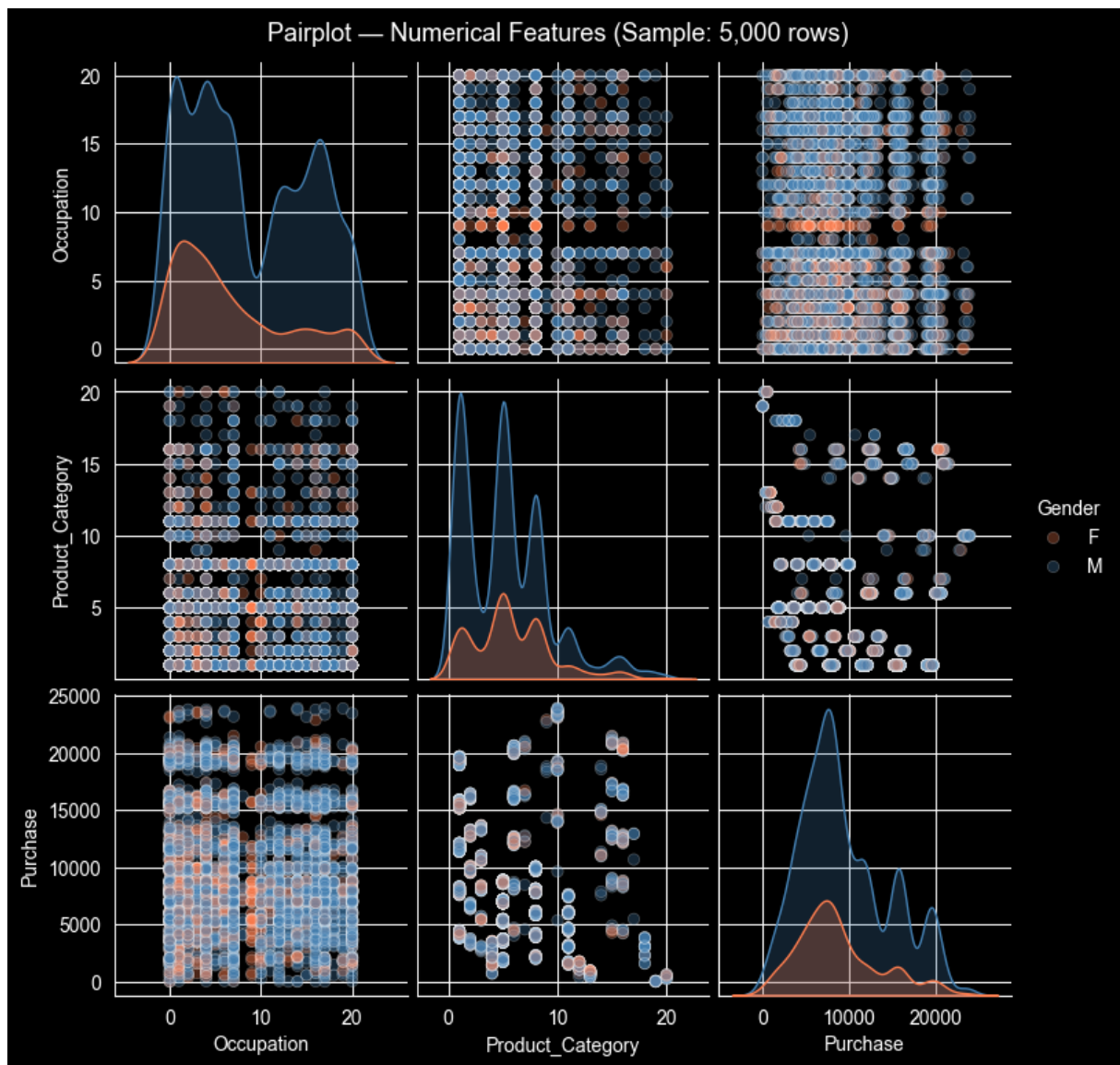


```

In [47]: # Pairplot — sample 5000 rows (550k is too slow to plot)
sample_df = df.sample(5000, random_state=42)

g = sns.pairplot(sample_df[['Occupation', 'Product_Category', 'Purchase']],
                 hue='Gender',
                 palette={'M': 'steelblue', 'F': 'coral'},
                 plot_kws={'alpha': 0.3},
                 diag_kind='kde')
g.fig.suptitle('Pairplot — Numerical Features (Sample: 5,000 rows)',
               fontsize=13, y=1.02)
plt.show()

```

Business Insights from EDA

Non-Graphical Observations:

- Gender: Dataset is male-dominated — 75% male (414,259) vs 25% female (135,809) transactions. This imbalance is in *transaction count*, not customer count.
- Age: 26–35 is the most active group (219,587 transactions = 40% of data), followed by 36–45 and 18–25.
- City: Tier B cities lead with 42% of transactions, suggesting Walmart's strongest urban middle market.
- Marital Status: 59% Single (0), 41% Married (1).
- Stay in City: 1 year is most common — many customers are relatively new residents.

Univariate Visual Observations:

- Purchase distribution is right-skewed with a long tail — consistent with our earlier mean (9,264) > *median*(8,047) finding.
- Product Category 1, 5, and 8 dominate transaction counts.

Bivariate Visual Observations:

- Gender vs Purchase: Male and female purchase distributions are visually similar in spread — the actual mean difference needs CLT analysis to confirm significance.
- Age vs Purchase: The 51–55 age group shows the highest median purchase. The 0–17 group has the lowest — expected given limited income.
- City Category: City C customers show wider purchase spread than A or B.
- Marital Status: Single and Married customers show nearly identical purchase distributions.

Correlation Observations:

- Notable finding: Product_Category and Purchase show a -0.344 correlation — the strongest relationship in the dataset. Lower-numbered categories (1, 5, 8) tend to have higher purchase amounts.
- All other numerical variables are near-zero correlated with Purchase.
- This confirms group-level analysis (CLT + CI) is the right approach.

4. Central Limit Theorem & Confidence Interval Analysis

Core Concept: The CLT states that regardless of the shape of the original distribution, the distribution of sample means approaches a normal distribution as sample size (n) increases. This allows us to build Confidence Intervals (CIs), a range within which the true population mean is very likely to lie.

Formula used throughout this section:

- Standard Error (SE) = Sample_Std / sqrt(Population_Size)
- CI = Sample_Mean ± (z × SE)
- z = 1.645 (90%) | 1.96 (95%) | 2.576 (99%)
- Population size assumed = 50,000,000 (per the business problem)

Question 1: Are Women Spending More Per Transaction Than Men?

```
In [50]: # Split data by gender
```

```

female_df = df[df['Gender'] == 'F']
male_df   = df[df['Gender'] == 'M']

female_mean = female_df['Purchase'].mean()
male_mean   = male_df['Purchase'].mean()
female_std  = female_df['Purchase'].std(ddof=1)
male_std    = male_df['Purchase'].std(ddof=1)

comparison = pd.DataFrame({
    'Female': [
        f"{len(female_df):,}",
        f"{female_mean:.2f}",
        f"{female_df['Purchase'].median():.2f}",
        f"{female_std:.2f}",
        f"{female_df['Purchase'].min():,}",
        f"{female_df['Purchase'].max():,}"
    ],
    'Male': [
        f"{len(male_df):,}",
        f"{male_mean:.2f}",
        f"{male_df['Purchase'].median():.2f}",
        f"{male_std:.2f}",
        f"{male_df['Purchase'].min():,}",
        f"{male_df['Purchase'].max():,}"
    ]
}, index=['Sample Size', 'Mean Purchase', 'Median Purchase',
        'Std Deviation', 'Min', 'Max'])

print("Gender-wise Purchase Statistics \n")
display(comparison)

diff = male_mean - female_mean
print(f"\nDifference (Male - Female): {diff:.2f}")
print(f"Men spend {(diff/female_mean*100):.1f}% more per transaction t")
print(f"\nPreliminary answer: NO – men spend MORE than women on averag")
print(f"But is this difference real for all 50M customers? -> CLT anal")

```

Gender-wise Purchase Statistics

	Female	Male
Sample Size	135,809	414,259
Mean Purchase	8734.57	9437.53
Median Purchase	7914.00	8098.00
Std Deviation	4767.23	5092.19
Min	12	12
Max	23,959	23,961

Difference (Male – Female): 702.96

Men spend 8.0% more per transaction than women on average.

Preliminary answer: NO – men spend MORE than women on average.

But is this difference real for all 50M customers? -> CLT analysis will confirm.

Central Limit Theorem -> Demonstration

As sample size (n) increases, the distribution of sample means:

1. Becomes more **bell-shaped** (normal) -> regardless of the original skewed distribution
2. Becomes **narrower** -> the mean is estimated more precisely
3. Stays **centered** at the true population mean

This is exactly why we can use sample data to make conclusions about 50 million customers.

```
In [51]: np.random.seed(42)
sample_sizes = [30, 100, 500, 1000, 5000]
n_iterations = 1000

fig, axes = plt.subplots(2, 5, figsize=(22, 8))

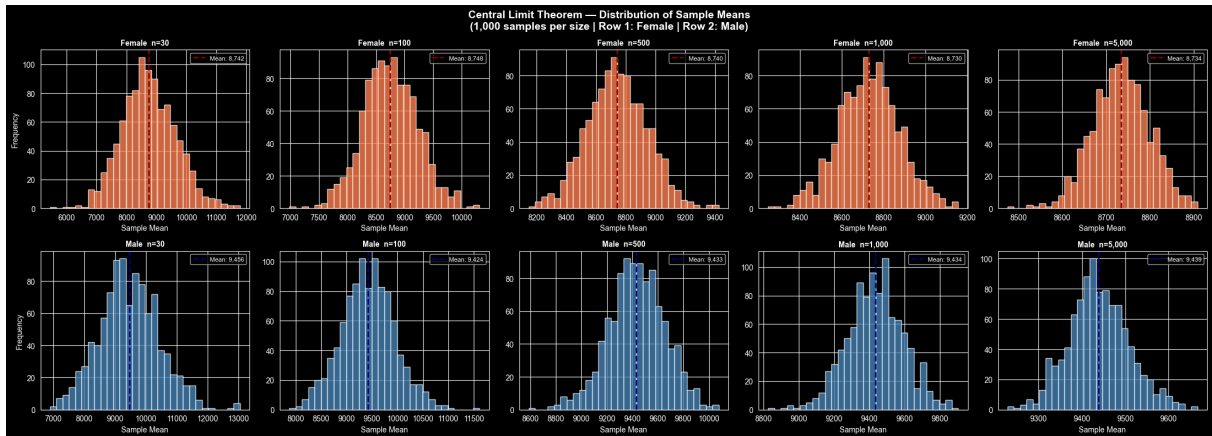
for idx, n in enumerate(sample_sizes):
    f_means = [female_df['Purchase'].sample(n).mean() for _ in range(n_iterations)]
    m_means = [male_df['Purchase'].sample(n).mean() for _ in range(n_iterations)]

    # Female row
    axes[0, idx].hist(f_means, bins=30, color='coral', alpha=0.8, edgecolor='black')
    axes[0, idx].axvline(np.mean(f_means), color='darkred', linestyle='solid',
                        label=f'Mean: {np.mean(f_means):.0f}')
    axes[0, idx].set_title(f'Female n={n:,}', fontsize=10, fontweight='bold')
    axes[0, idx].set_xlabel('Sample Mean')
    axes[0, idx].legend(fontsize=8)
    if idx == 0: axes[0, idx].set_ylabel('Frequency')

    # Male row
    axes[1, idx].hist(m_means, bins=30, color='steelblue', alpha=0.8, edgecolor='black')
    axes[1, idx].axvline(np.mean(m_means), color='darkblue', linestyle='solid',
                        label=f'Mean: {np.mean(m_means):.0f}')
    axes[1, idx].set_title(f'Male n={n:,}', fontsize=10, fontweight='bold')
    axes[1, idx].set_xlabel('Sample Mean')
    axes[1, idx].legend(fontsize=8)
    if idx == 0: axes[1, idx].set_ylabel('Frequency')

plt.suptitle('Central Limit Theorem – Distribution of Sample Means\n'
            '(1,000 samples per size | Row 1: Female | Row 2: Male)',
            fontsize=14, fontweight='bold')
```

```
plt.tight_layout()
plt.show()
```



Confidence Intervals — Male vs Female

```
In [66]: # Population size given in the problem
n_population = 50_000_000

# CI formula: mean +/- z * (std / sqrt(n))
def compute_ci(data, n_pop, confidence):
    sample_mean = data.mean()
    sample_std = data.std(ddof=1)
    std_error = sample_std / np.sqrt(n_pop)

    # z-scores for each confidence level
    z_values = {0.90: 1.645, 0.95: 1.96, 0.99: 2.576}
    z = z_values[confidence]

    margin = z * std_error
    lower = sample_mean - margin
    upper = sample_mean + margin

    return {'Mean': round(sample_mean, 4), 'Std': round(sample_std, 4),
            'SE': round(std_error, 6), 'Z': z, 'Margin': round(margin,
            'Lower': round(lower, 4), 'Upper': round(upper, 4)}

# Female customers
print("Female Customers - Population 50 Million")
for conf in [0.90, 0.95, 0.99]:
    r = compute_ci(female_df['Purchase'], n_population, conf)
    print(f"{int(conf*100)}% CI : ({r['Lower']}, {r['Upper']}) Mean:

print()

# Male customers
print("Male Customers - Population 50 Million")
for conf in [0.90, 0.95, 0.99]:
    r = compute_ci(male_df['Purchase'], n_population, conf)
    print(f"{int(conf*100)}% CI : ({r['Lower']}, {r['Upper']}) Mean:
```

Female Customers – Population 50 Million

90% CI : (8733.4567, 8735.6748) Mean: 8734.5658

95% CI : (8733.2444, 8735.8872) Mean: 8734.5658

99% CI : (8732.8291, 8736.3025) Mean: 8734.5658

Male Customers – Population 50 Million

90% CI : (9436.3414, 9438.7107) Mean: 9437.526

95% CI : (9436.1146, 9438.9375) Mean: 9437.526

99% CI : (9435.6709, 9439.3811) Mean: 9437.526

```
In [54]: fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for idx, conf in enumerate([0.90, 0.95, 0.99]):
    f_ci = compute_ci(female_df['Purchase'], n_population, conf)
    m_ci = compute_ci(male_df['Purchase'], n_population, conf)

    means = [f_ci['Mean'], m_ci['Mean']]
    margins = [f_ci['Margin'], m_ci['Margin']]

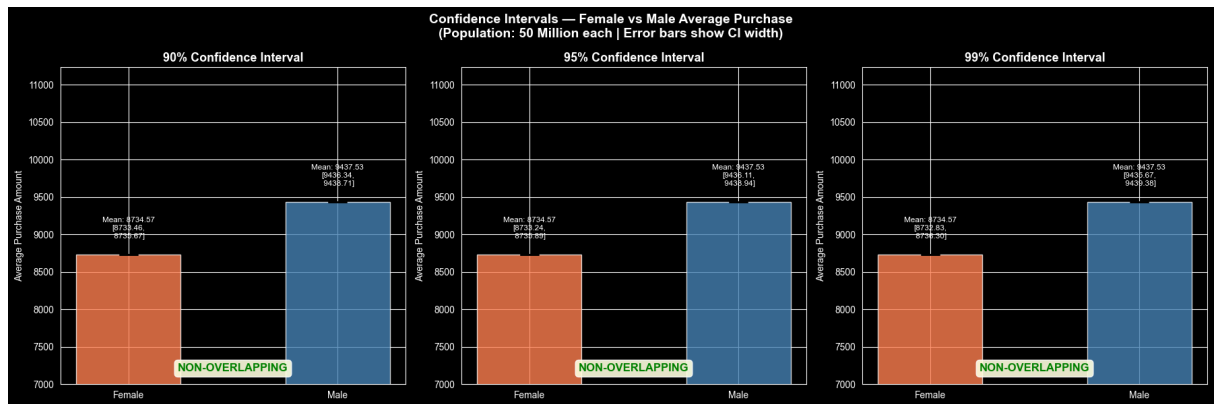
    bars = axes[idx].bar(['Female', 'Male'], means,
                        color=['coral', 'steelblue'], alpha=0.8, width
                        yerr=margins, capsize=10,
                        error_kw={'elinewidth':2, 'capthick':2, 'eco

    for bar, ci in zip(bars, [f_ci, m_ci]):
        axes[idx].text(bar.get_x() + bar.get_width()/2, ci['Mean'] + 2
                        f"Mean: {ci['Mean']:.2f}\n[{ci['Lower']:.2f},\n
                        ha='center', va='bottom', fontsize=8.5)

    axes[idx].set_title(f'{int(conf*100)}% Confidence Interval', fonts
    axes[idx].set_ylabel('Average Purchase Amount')
    axes[idx].set_ylim(7000, max(means) + 1800)

    overlap = f_ci['Upper'] >= m_ci['Lower']
    status = "OVERLAPPING" if overlap else "NON-OVERLAPPING"
    col = 'red' if overlap else 'green'
    axes[idx].text(0.5, 0.04, status, transform=axes[idx].transAxes,
                  ha='center', fontsize=12, color=col, fontweight='bo
                  bbox=dict(boxstyle='round', facecolor='lightyellow'

plt.suptitle('Confidence Intervals – Female vs Male Average Purchase\n
              (Population: 50 Million each | Error bars show CI width)
              fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



Are the CIs Overlapping? Business Implications

Finding:

- The confidence intervals for male and female average purchase amounts are **NON-OVERLAPPING at all confidence levels (90%, 95%, 99%)**.
- The gap between the upper bound of female CI and lower bound of male CI is approximately 700 units — while each CI is only $\pm 1-2$ units wide.
- This is a statistically significant result. We can say with 99% confidence that the true average male purchase is higher than the true average female purchase.

Why are the CIs so narrow? Because we are projecting to a population of 50 million customers. $SE = Std / \sqrt{50,000,000}$ — the denominator is huge, making SE tiny. This means even a small difference in means is highly significant at population scale.

Business Implications for Walmart:

1. Men consistently and significantly spend more per Black Friday transaction. This is not a sampling fluke — it holds for the entire population.
2. Walmart should design gender-targeted promotions to lift female spending. Even closing 25% of the gap (from 8,734 to 8,909) across 50M female customers adds billions in revenue.
3. High-value product placement and bundling strategies should differ by gender.
4. Female customers may prefer different product categories — Walmart should analyse which categories drive female purchases and promote them aggressively.

Married vs Unmarried — Confidence Interval Analysis

```
In [57]: single_df = df[df['Marital_Status'].astype(str) == '0']
married_df = df[df['Marital_Status'].astype(str) == '1']

print(" Marital Status :- Purchase Statistics \n")
for name, grp in [('Single (0)', single_df), ('Married (1)', married_d
    print(f"{name}: n={len(grp):,} | Mean={grp['Purchase'].mean():.2f}
          f"Std={grp['Purchase'].std():.2f}")

print("\n Confidence Intervals ")
for name, grp in [('Single', single_df), ('Married', married_df)]:
    print(f"\n{name}:")
    for conf in [0.90, 0.95, 0.99]:
        r = compute_ci(grp['Purchase'], n_population, conf)
        print(f"    {int(conf*100)}% CI: [{r['Lower']:.4f}, {r['Upper']:.4f}]
              f"Mean: {r['Mean']:.4f}")

# Overlap check
s_ci = compute_ci(single_df['Purchase'], n_population, 0.95)
m_ci = compute_ci(married_df['Purchase'], n_population, 0.95)
overlap = s_ci['Upper'] >= m_ci['Lower'] or m_ci['Upper'] >= s_ci['Low
print(f"\n95% CI Overlap: {'YES - NOT significant' if overlap else 'NO
print(f"Single mean: {s_ci['Mean']:.2f} | Married mean: {m_ci['Mean']:.2f}
print(f"Difference: {abs(s_ci['Mean'] - m_ci['Mean']):.2f}")
```

Marital Status :- Purchase Statistics

Single (0): n=324,731 | Mean=9265.91 | Std=5027.35
 Married (1): n=225,337 | Mean=9261.17 | Std=5016.90

Confidence Intervals

Single:

90% CI:	[9264.7381, 9267.0772]	Mean: 9265.9076
95% CI:	[9264.5141, 9267.3011]	Mean: 9265.9076
99% CI:	[9264.0761, 9267.7391]	Mean: 9265.9076

Married:

90% CI:	[9260.0075, 9262.3417]	Mean: 9261.1746
95% CI:	[9259.7840, 9262.5652]	Mean: 9261.1746
99% CI:	[9259.3469, 9263.0022]	Mean: 9261.1746

95% CI Overlap: YES - NOT significant

Single mean: 9265.91 | Married mean: 9261.17

Difference: 4.73

```
In [59]: fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for idx, conf in enumerate([0.90, 0.95, 0.99]):
    s_ci = compute_ci(single_df['Purchase'], n_population, conf)
    m_ci = compute_ci(married_df['Purchase'], n_population, conf)

    means = [s_ci['Mean'], m_ci['Mean']]
    margins = [s_ci['Margin'], m_ci['Margin']]
```



```

bars = axes[idx].bar(['Single', 'Married'], means,
                    color=['#2ecc71', '#9b59b6'], alpha=0.8, width
                    yerr=margins, capsize=10,
                    error_kw={'elinewidth':2, 'capthick':2, 'ecol

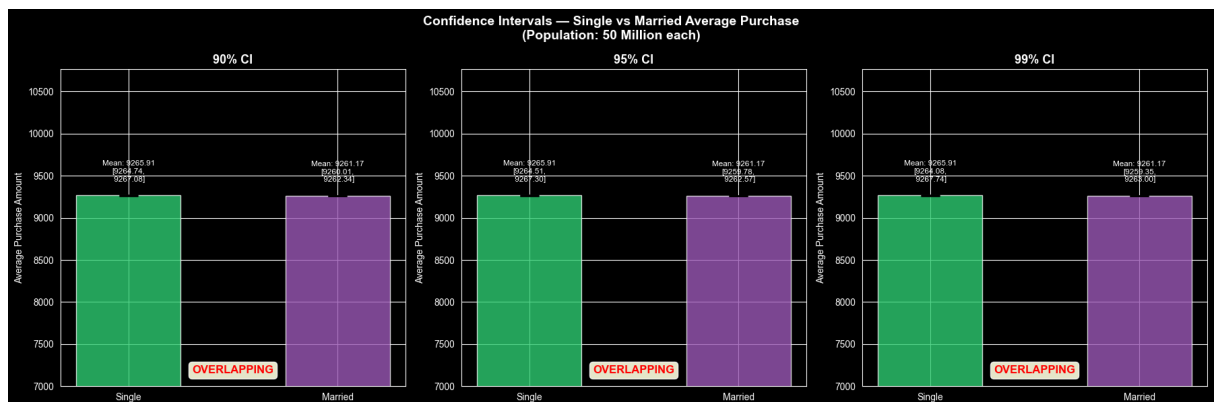
for bar, ci in zip(bars, [s_ci, m_ci]):
    axes[idx].text(bar.get_x() + bar.get_width()/2, ci['Mean'] + 1
                  f'Mean: {ci['Mean']:.2f}\n[{ci['Lower']:.2f},\n
                  ha='center', va='bottom', fontsize=8.5)

axes[idx].set_title(f'{int(conf*100)}% CI', fontsize=13, fontweigh
axes[idx].set_ylabel('Average Purchase Amount')
axes[idx].set_ylim(7000, max(means) + 1500)

overlap = s_ci['Upper'] >= m_ci['Lower'] or m_ci['Upper'] >= s_ci[
status = "OVERLAPPING" if overlap else "NON-OVERLAPPING"
col     = 'red' if overlap else 'green'
axes[idx].text(0.5, 0.04, status, transform=axes[idx].transAxes,
              ha='center', fontsize=12, color=col, fontweight='bo
              bbox=dict(boxstyle='round', facecolor='lightyellow'

plt.suptitle('Confidence Intervals – Single vs Married Average Purchas
              '(Population: 50 Million each)',
              fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```



Age Group Analysis — Confidence Interval Analysis

```

In [62]: age_order = ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']

print(" Age Group :- Purchase Statistics & 95% CIs \n")
age_results = {}
for age in age_order:
    grp = df[df['Age'].astype(str) == age]['Purchase']
    ci = compute_ci(grp, n_population, 0.95)
    age_results[age] = ci
    print(f"{age:<8}: n={len(grp):>6,} | Mean={ci['Mean']:>8.2f} | "
          f"95% CI: [{ci['Lower']:.2f}, {ci['Upper']:.2f}]")

```

Age Group :- Purchase Statistics & 95% CIs

0-17	:	n=15,102		Mean= 8933.46		95% CI: [8932.05, 8934.88]
18-25	:	n=99,660		Mean= 9169.66		95% CI: [9168.27, 9171.06]
26-35	:	n=219,587		Mean= 9252.69		95% CI: [9251.30, 9254.08]
36-45	:	n=110,013		Mean= 9331.35		95% CI: [9329.96, 9332.74]
46-50	:	n=45,701		Mean= 9208.63		95% CI: [9207.25, 9210.00]
51-55	:	n=38,501		Mean= 9534.81		95% CI: [9533.40, 9536.22]
55+	:	n=21,504		Mean= 9336.28		95% CI: [9334.89, 9337.67]

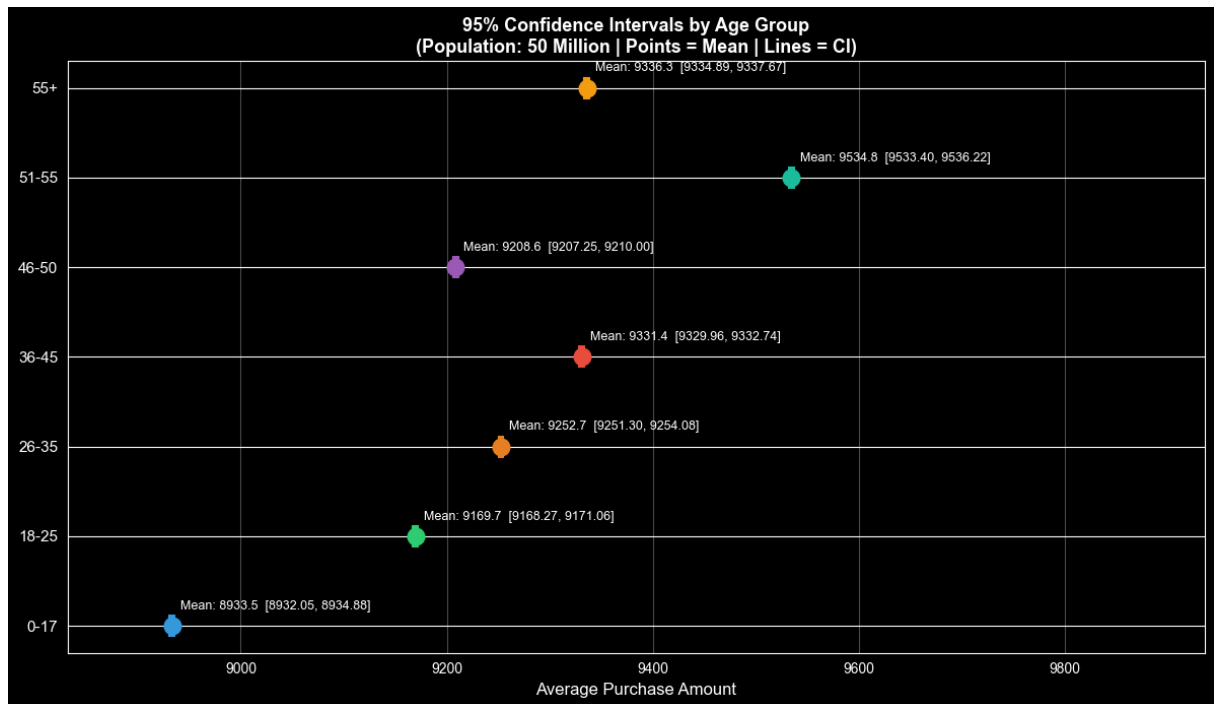
```
In [63]: fig, ax = plt.subplots(figsize=(12, 7))

colors_age = ['#3498db', '#2ecc71', '#e67e22', '#e74c3c', '#9b59b6', '#1abc9c']

for i, (age, color) in enumerate(zip(age_order, colors_age)):
    ci = age_results[age]
    ax.errorbar(ci['Mean'], i,
                xerr=[[ci['Mean'] - ci['Lower']], [ci['Upper'] - ci['Mean']]],
                fmt='o', color=color, capsize=8, capthick=3,
                elinewidth=3, markersize=12, label=age)
    ax.text(ci['Mean'] + 8, i + 0.15,
            f'Mean: {ci['Mean']:.1f}  [{ci['Lower']:.2f}, {ci['Upper']:.2f}]',
            va='bottom', fontsize=9)

ax.set_yticks(range(len(age_order)))
ax.set_yticklabels(age_order, fontsize=11)
ax.set_xlabel('Average Purchase Amount', fontsize=12)
ax.set_title('95% Confidence Intervals by Age Group\n'
             '(Population: 50 Million | Points = Mean | Lines = CI)',
             fontsize=13, fontweight='bold')
ax.grid(axis='x', alpha=0.3)
ax.set_xlim(min(r['Lower'] for r in age_results.values()) - 100,
            max(r['Upper'] for r in age_results.values()) + 400)

plt.tight_layout()
plt.show()
```



Observations :- Age Group Analysis

- Age groups have **non-overlapping CIs**, meaning spending differences across age groups are statistically significant at the population level.
- The **51-55 age group** has the highest average purchase :- likely due to higher disposable income in peak earning years.
- The **0-17 group** has the lowest average purchase :- expected given limited financial independence.
- The **26-35 group** has the highest transaction count (40% of all purchases) but is not the highest spender per transaction :- volume does not equal value.

Business Implication: Walmart should tailor Black Friday promotions differently by life stage:

- Younger shoppers (18-25): discount-driven, focus on price-sensitive products
- Prime earners (36-55): premium products, convenience, quality over price
- Seniors (55+): fewer transactions but moderate spend :- loyalty programs

5. Final Insights

Distribution and Variable Relationships

Purchase Amount Distribution:

- The raw purchase distribution is right-skewed (mean 9,264 > median 8,047). However, once we apply the CLT and look at the distribution of sample means, it becomes perfectly normal, this is the CLT in action.
- The skewness in raw data does NOT affect our population-level conclusions because we are working with sample means, not individual values.

Gender:

- Male customers generate 75% of all transactions but also spend significantly more per transaction (mean 9,437 vs 8,734 for females).
- This is NOT just a volume effect, the per-transaction spend is genuinely higher for men, confirmed by non-overlapping CIs at 99% confidence.
- Possible reason: men may purchase higher-priced electronics and gadgets, while women may buy more lower-priced household items.

Marital Status:

- Contrary to what one might expect, marital status has NO statistically significant effect on purchase amount. Single (9,265) vs Married (9,261) differ by just 4.73.
- The overlapping CIs confirm this is noise, not signal.
- Implication: Walmart should NOT segment its Black Friday strategy by marital status, the spending behavior is identical between both groups.

Age Groups:

- All age group CIs are non-overlapping, spending differs significantly across life stages.
- 51-55 spends the most (9,534), likely due to peak earning years and fewer financial obligations compared to 36-45 age group.
- 0-17 spends the least (8,933), expected given limited financial independence.
- Interestingly, 26-35 has the highest transaction COUNT but is not the highest spender per transaction. This age group drives volume, not value.

City Category:

- City B dominates transaction count (42%), Walmart's Tier 2 urban market is its largest customer base.
- City C shows the widest purchase spread, suggesting more diverse buying behavior in Tier 3 cities.

Product Category:

- The -0.344 correlation between Product_Category and Purchase indicates that lower-numbered categories (1, 5, 8) are higher-value product lines.
- These categories should receive premium placement during Black Friday campaigns.

Generalizing to the Population of 50 Million Customers

Using the CLT, we have established the following with 99% confidence:

Group	Population Mean CI (99%)	Significant?
Female (50M)	[8732.83, 8736.30]	—
Male (50M)	[9435.67, 9439.38]	YES — higher than female
Single (50M)	[9264.08, 9267.74]	—
Married (50M)	[9259.35, 9263.00]	NO — overlaps with single
Age 51-55 (50M)	[9533.40, 9536.22]	YES — highest age group
Age 0-17 (50M)	[8932.05, 8934.88]	YES — lowest age group

Key takeaway: The most reliable predictor of higher Black Friday spending is being male and being in the 46-55 age bracket. Marital status is irrelevant.

6. Business Recommendations

Based on the CLT and Confidence Interval analysis of 550,068 Black Friday transactions, here are actionable recommendations for Walmart leadership:

Recommendation 1 :- Close the Gender Spending Gap

Men spend 703 more per transaction than women on average, confirmed for the entire population. Walmart should create female-focused Black Friday deals on categories women buy most (household, apparel, beauty). Even a 10% increase in female spend per transaction across 50M customers would generate billions in additional revenue.

Recommendation 2 :- Invest Heavily in the 46-55 Age Segment

The 51-55 and 46-50 age groups are the highest spenders per transaction. Walmart should offer premium product bundles, early access deals, and loyalty rewards targeted at this segment. These customers have the budget and

willingness to spend.

Recommendation 3 :- Drive Volume from the 26-35 Segment

The 26-35 group makes the most transactions (40% of all purchases) but does not spend the most per transaction. Upselling and bundling strategies during checkout, such as "frequently bought together" offers, can increase their basket size.

Recommendation 4 :- Do Not Segment by Marital Status

Analysis confirms that single and married customers spend virtually the same amount. Any marketing budget allocated to marital-status-based campaigns would be wasted. Redirect that budget to gender and age-based targeting instead.

Recommendation 5 :- Focus Black Friday Campaigns on Tier B Cities

City B generates 42% of all transactions. Walmart should concentrate its Black Friday promotional spend, inventory stocking, and staffing in Tier B cities to maximise return on investment.

Recommendation 6 :- Promote High-Value Product Categories Prominently

Product Categories 1, 5, and 8 drive the highest transaction counts and correlate with higher purchase amounts. These should receive the best floor placement, homepage banner ads online, and the deepest promotional discounts to drive maximum revenue during Black Friday.

In []: